



Informe Tarea 1: Base de Datos Avanzadas

Felipe Santiago Parra Díaz

Rol: 202373568-k

Rut: 21.758.067-6

Paralelo: 200

Link Github: <https://github.com/ShadierPrune/BD-Av-Tarea-1-MongoDB.git>

Consideraciones:

- Se uso Windows 11 para realizar la tarea.
- Las imágenes del proceso están almacenadas en el Link Github: <https://github.com/ShadierPrune/BD-Av-Tarea-1-MongoDB.git>. Importante he de considerar que en dicho github se almacena cómo se realiza la consulta con MongoDB Compass y luego como sería por terminal, además de mostrar algunos resultados en las imágenes y en archivos .json la respuesta completa.
- En este informe se mostrarán algunas de las respuestas y no todas, se decidió hacerlo así para no molestar a la lectura, pero en el github se encuentran las respuestas a cada consulta de forma ordenada en carpetas: **consultas/Parte-X/consulta-Y/** en donde se encuentran archivos .json con la respuesta de dicha consulta.

Preparación de los datos

Para generar los datos se pensó en utilizar un generador de Python, pero al tomar en consideración que el peso e importancia de la tarea se encuentra en el uso de las consultas, se prefirió utilizar IA, tal como menciona el enunciado de la tarea que se puede. Utilice el siguiente prompt en ChatGPT:

Chat, necesito crear una colección de datos en formato .json, quiero que sea una lista de los siguientes actores o actores de voz, que tengan los siguientes campos:

- *Nombre: nombre completo de la persona (actor o actriz) (con espacios incluidos, ejemplo: “Juan Pérez Díaz”)*
- *Nacionalidad: país donde nació*
- *Fecha de nacimiento: formato AÑO/MES/DIA*
- *Edad: última edad registrada (si está muerto dejar la edad a la que murió)*
- *Patrimonio: patrimonio estimado de la persona (para generalizar use dolares)*
- *Idiomas: arreglo simple que indique uno o más idiomas en los que la persona ha realizado trabajos de doblaje. Por ejemplo: [“Español”, “Inglés”]*
- *Personajes: arreglo compuesto por documentos embebidos. Cada personaje deberá contener, al menos:*
 - a) *Nombre del personaje*
 - b) *Serie, película, anime o videojuego al que pertenece.*
 - c) *Tipo de producción: profesional o independiente.*



d) Rol: principal, secundario o extra.

e) Cantidad de apariciones. (número entero)

f) Géneros de la producción, almacenados como un arreglo simple, por ejemplo:
["Terror", "Comedia"]

A partir de estos campos necesito que crees una colección de al menos 30 actores o actrices normales o de voz. Me gustaría que uses estos valores de ejemplo y que tengan sentido:

nombres_personajes = [

"Goku", "Gohan", "Naruto", "Sakura", "Harry Potter", "Sung Jin-woo",

"Ben Tennyson", "Hawkeye", "Simon Belmont", "Trevor Belmont",

"Ritcher Belmont", "Kim Gong-Ja", "Peter Parker", "Mr. Darcy",

"Elizabeth Bennet", "William Thacker", "Anna Scott", "Jonesy Jones",

"Alexander Cooper", "Ben Cooper", "Kelly Cooper", "Vegeta"

]

titulos_obras = [

"Dragon Ball Z", "Naruto", "Ben 10", "El último de los Mohicanos",

"CastleVania", "Soy leyenda", "Cazador Suicida de Clase SSS",

"Solo Leveling", "Misión Imposible III", "Rápidos y Furiosos 8",

"Spider-Man", "Detective Pikachu", "Orgullo y Prejuicio",

"Notting Hill", "Fortnite",

"Alexander y un día terrible, horrible, malo... ¡Muy malo!"

]

Estos son solo ejemplos que podrías ocupar, procura que tengan sentido y PUEDES TOMAR TODO EL TIEMPO QUE NECESITES EN CREAMLOS, no olvides crear 30 actores por favor.

Después de eso lo generó sin el campo Géneros de la producción, se lo pedí de nuevo y me devolvió lo siguiente:



Descargar el JSON actualizado

Compartir ...

La estructura queda así:

JSON

```
{
  "Nombre": "Mario Castañeda",
  "Nacionalidad": "México",
  "Fecha de nacimiento": "1962/06/29",
  "Edad": 64,
  "Patrimonio": 3000000,
  "Idiomas": ["Español"],
  "Personajes": [
    {
      "Nombre": "Goku",
      "Serie, película, anime o videojuego": "Dragon Ball Z",
      "Tipo de producción": "profesional",
      "Rol": "principal",
      "Cantidad de apariciones": 291,
      "Géneros": [
        "Acción",
        "Aventura",
        "Fantasía",
        "Artes marciales"
      ]
    }
  ]
}
```

Así ya queda con la estructura adecuada para cargarlo como **colección de documentos en MongoDB**, incluyendo los arreglos y documentos embebidos que necesitabas.

actores_actrices_doblaie (1)

... actores_actrices_doblaie.json

+ Pregunta lo que quieras

Pensar



Parte 1: Consultas:

Para las consultas revisé que el orden del `.find` es (filtro, project) y lo que se quiera añadir después parece que se hace con `.sort .limit` ya que es algo “aparte” de la consulta.

Consulta 1: Proyección, filtrado y ordenamiento sobre campos simples.

```
.find(  
  {"Edad": {$gt:50}},  
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})  
  .sort({"Edad": 1})  
  .limit(10);
```

```
>_MONGOSH  
> db["Voice-actors"].find(  
  {"Edad":{$gt:50}},  
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})  
  .sort({"Edad": 1})  
  .limit(10);  
  
< {  
  Nombre: 'René García',  
  Nacionalidad: 'México',  
  Edad: 52,  
  Personajes: [  
    {  
      Nombre: 'Vegeta',  
      'Serie, película, anime o videojuego': 'Dragon Ball Z',  
      'Tipo de producción': 'profesional',  
      Rol: 'principal',  
      'Cantidad de apariciones': 291,  
      'Géneros': [  
        'Acción',  
        'Aventura',  
        'Fantasía',  
        'Artes marciales'  
      ]  
    },  
    {  
      Nombre: 'Vegeta',  
      'Serie, película, anime o videojuego': 'Dragon Ball Super',  
      'Tipo de producción': 'profesional',  
      Rol: 'principal',  
      'Cantidad de apariciones': 131
```



Consulta 2: Búsqueda dentro de un arreglo simple

```
.find(  
  {"Nacionalidad": "Estados Unidos","Idiomas":{"$in: ["Inglés"]}},  
  {"_id": 0, "Nombre":1,"Idiomas": 1, "Nacionalidad":1,"Edad": 1})  
.sort({"Nombre":1})  
.limit(10)
```

The screenshot shows a MongoDB shell window with the following tabs: 'Welcome', 'Voice-actors', and 'mongosh: Tarea-1-BD-Av'. The command prompt is 'x_MONGOSH'. The user has entered the following commands:

```
> use("Actors")  
< switched to db Actors  
> db["Voice-actors"].find(  
  {"Nacionalidad": "Estados Unidos","Idiomas":{"$in: ["Inglés"]}},  
  {"_id": 0, "Nombre":1,"Idiomas": 1, "Nacionalidad":1,"Edad": 1})  
  .sort({"Nombre":1})  
  .limit(10)
```

The output shows three documents from the 'Voice-actors' collection, sorted by 'Nombre' in ascending order:

```
< {  
  Nombre: 'Aleks Le',  
  Nacionalidad: 'Estados Unidos',  
  Edad: 27,  
  Idiomas: [  
    'Inglés'  
  ]  
}  
{  
  Nombre: 'Chris Evans',  
  Nacionalidad: 'Estados Unidos',  
  Edad: 45,  
  Idiomas: [  
    'Inglés'  
  ]  
}  
{  
  Nombre: 'Dwayne Johnson',  
  Nacionalidad: 'Estados Unidos',  
  Edad: 54,  
  Idiomas: [  
    'Inglés'
```

Consulta 3: Búsqueda sobre arreglos de documentos embebidos

```
.find(  
  {"Edad":{$lte: 30},"Personajes":{$elemMatch:{"Rol":"principal", "Géneros":{$in:  
["Drama"]}}} }},  
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes": 1})  
.sort({"Nacionalidad":1})  
.limit(10)
```

```
>_MONGOSH  
> use("Actors")  
< switched to db Actors  
> db["voice-actors"].find(  
  {"Edad":{$lte: 30},"Personajes":{$elemMatch:{"Rol":"principal", "Géneros":{$in: ["Drama"]}}} },  
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes": 1})  
  .sort({"Nacionalidad":1})  
  .limit(10)  
< {  
  Nombre: 'Aleks Le',  
  Nacionalidad: 'Estados Unidos',  
  Edad: 27,  
  Personajes: [  
    {  
      Nombre: 'Mikey',  
      'Serie, película, anime o videojuego': 'Tokyo Revengers',  
      'Tipo de producción': 'profesional',  
      Rol: 'principal',  
      'Cantidad de apariciones': 50,  
      'Géneros': [  
        'Acción',  
        'Drama',  
        'Artes marciales'  
      ]  
    },  
  ],  
  {  
    Nombre: 'Zenitsu Agatsuma',  
    'Serie, película, anime o videojuego': 'Demon Slayer',  
    'Tipo de producción': 'profesional',  
    Rol: 'secundario',
```



Consulta 4: Pipeline de agregación.

```
.aggregate([  
  {  
    $project: {  
      Nombre: 1,  
      _id: 0,  
      Patrimonio: 1,  
      Edad: 1,  
      Nacionalidad: 1  
    }  
  },  
  {  
    $sort: {Patrimonio: -1}  
  },  
  {$skip: 4},  
  {$limit: 10}  
])
```

```
> _MONGOSH  
> use("Actors")  
< switched to db Actors  
> db["Voice-actors"].aggregate([  
  {  
    $project: {  
      Nombre: 1,  
      _id: 0,  
      Patrimonio: 1,  
      Edad: 1,  
      Nacionalidad: 1  
    }  
  },  
  {  
    $sort: {Patrimonio: -1}  
  },  
  {$skip: 4},  
  {$limit: 10}  
])  
< [ {  
  Nombre: 'Hugh Grant',  
  Nacionalidad: 'Reino Unido',  
  Edad: 66,  
  Patrimonio: 150000000  
}, {  
  Nombre: 'Daniel Radcliffe',  
  Nacionalidad: 'Reino Unido',  
  Edad: 37,  
  Patrimonio: 110000000  
}]
```

Parte 2: Procesamiento de consultas

1) Ejecución sin índices

Para las consultas 1, 2 y 3 solo se debe de añadir al final de cada consulta un:

`.explain("executionStats")`

Consulta 1: Proyección, filtrado y ordenamiento sobre campos simples

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Edad":{"$gt":50}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      Edad: {
        '$gt': 50
      }
    },
    indexFilterSet: false,
    queryHash: 'A7570537',
    planCacheShapeHash: 'A7570537',
    planCacheKey: 'B2DADEBB',
    optimizationTimeMillis: 2,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'PROJECTION_SIMPLE',
      transformBy: {
```

Consulta 2: Búsqueda dentro de un arreglo simple

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Nacionalidad": "Estados Unidos","Idiomas":{"$in: ["Inglés"]}},
  {"_id": 0, "Nombre":1,"Idiomas": 1, "Nacionalidad":1,"Edad": 1})
.sort({"Nombre":1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Idiomas: {
            '$eq': 'Inglés'
          }
        },
        {
          Nacionalidad: {
            '$eq': 'Estados Unidos'
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: 'A2EC8B8A',
    planCacheShapeHash: 'A2EC8B8A',
    planCacheKey: '23A50762',
```

Consulta 3: Búsqueda sobre arreglos de documentos embebidos

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Edad":{"$lte: 30},"Personajes":{"$elemMatch":{"Rol":"principal", "Géneros":{"$in: ["Drama"]}} }},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes": 1})
.sort({"Nacionalidad":1})
.limit(10).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Personajes: {
            '$elemMatch': {
              '$and': [
                {
                  Géneros: {
                    '$eq': 'Drama'
                  }
                },
                {
                  Rol: {
                    '$eq': 'principal'
                  }
                }
              ]
            }
          }
        }
      ]
    }
  }
}
```

Consulta 4: Pipeline de agregación

Para conseguir esta información se debe de hacer click en la parte que dice “**Explain**” a arriba a la derecha de la sección “**Aggregations**”.

Explain Plan

Explain provides key execution metrics that help diagnose slow queries and optimize index usage. [Learn more](#)

Visual Tree Raw Output

```
{
  "explainVersion": "1",
  "stages": [
    {
      "$cursor": {
        "queryPlanner": {
          "namespace": "Actors.Voice-actors",
          "parsedQuery": {},
          "indexFilterSet": false,
          "queryHash": "13DD9E3D",
          "planCacheShapeHash": "13DD9E3D",
          "planCacheKey": "5C6E11E9",
          "optimizationTimeMillis": 0,
          "maxIndexedOrSolutionsReached": false,
          "maxIndexedAndSolutionsReached": false,
          "maxScansToExplodeReached": false,
          "prunedSimilarIndexes": false,
          "winningPlan": {
            "isCached": false,
            "stage": "PROJECTION_SIMPLE",
            "transformBy": {
              "Nombre": true,
              "Patrimonio": true,
              "Edad": true,
              "Nacionalidad": true,
              "_id": false
            }
          },
          "inputStage": {

```

Query Performance Summary

- 10 documents returned
- 35 documents examined
- 2 ms execution time
- Is not sorted in memory
- 0 index keys examined
- No index available for this query.

Close

2) Índice sobre Nacionalidad

Para crear el índice use: `.createIndex({"Nacionalidad":1})`, lo cual genera un índice que los ordena de menor a mayor.

Consulta 1:

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Edad":{"$gt":50}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      Edad: {
        '$gt': 50
      }
    },
    indexFilterSet: false,
    queryHash: 'A757D537',
    planCacheShapeHash: 'A757D537',
    planCacheKey: 'B2DADEBB',
    optimizationTimeMillis: 6,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'PROJECTION_SIMPLE',
      transformBy: {
```

Consulta 2:

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Nacionalidad": "Estados Unidos","Idiomas":{"$in: ["Inglés"]}},
  {"_id": 0, "Nombre":1,"Idiomas": 1, "Nacionalidad":1,"Edad": 1})
.sort({"Nombre":1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Idiomas: {
            '$eq': 'Inglés'
          }
        },
        {
          Nacionalidad: {
            '$eq': 'Estados Unidos'
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: 'A2EC8B8A',
    planCacheShapeHash: 'A2EC8B8A',
    planCacheKey: 'A2EC8B8A',
    optimizationTimeMillis: 6,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'PROJECTION_SIMPLE',
      transformBy: {
```



Consulta 3

```
>_MONGOSH
> use("Actors")
< switched to db Actors
> db["Voice-actors"].find(
  {"Edad":{"$lte: 30},"Personajes":{"$elemMatch":{"Rol":"principal", "Géneros":{"$in: ["Drama"]}}}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes": 1})
.sort({"Nacionalidad":1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Personajes: {
            '$elemMatch': {
              '$and': [
                {
                  Géneros: {
                    '$eq': 'Drama'
                  }
                },
                {
                  Rol: {
                    '$eq': 'principal'
                  }
                }
              ]
            }
          }
        }
      ]
    }
  }
}
```

Consulta 4

Explain Plan

Explain provides key execution metrics that help diagnose slow queries and optimize index usage. [Learn more](#)

Visual Tree **Raw Output**

```
{
  "explainVersion": "1",
  "stages": [
    {
      "$cursor": {
        "queryPlanner": {
          "namespace": "Actors.Voice-actors",
          "parsedQuery": {},
          "indexFilterSet": false,
          "queryHash": "130D9E3D",
          "planCacheKey": "130D9E3D",
          "planCacheKey": "5C6E11E9",
          "optimizationTimeMillis": 6,
          "maxIndexedOrSolutionsReached": false,
          "maxIndexedAndSolutionsReached": false,
          "maxScansToExploreReached": false,
          "prunedSimilarIndexes": false,
          "winningPlan": {
            "isCached": false,
            "stage": "PROJECTION_SIMPLE",
            "transformBy": {
              "Nombre": true,
              "Patrimonio": true,
              "Edad": true,
              "Nacionalidad": true,
              "_id": false
            },
            "inputStage": {

```

Query Performance Summary

- 10 documents returned
- 35 documents examined
- 1 ms execution time
- Is not sorted in memory
- 0 index keys examined
- No index available for this query.

Close

Comparación después de añadir el índice de Nacionalidad

Comparación consulta 1:

Campo	Antes	Después	Conclusión
totalDocsExamined	35	35	Es casi igual pero toma más tiempo, esto puede deberse a que se añadió un nuevo campo por el
totalKeysExamined	0	0	
nReturned (global)	10	10	

nReturned (inputStage)	21	21	índice, y este puede hacer que me demore más en buscar. Veo la misma cantidad de documentos, pero ahora cada documento es más “grande” y por eso se demora más
executionTimeMillis	5	11	
Plan usado/elegido	PROJECTION_SIMPLE		

Comparación consulta 2:

Campo	Antes	Después	Conclusión
totalDocsExamined	35	12	Lee menos documentos, pero usa más llaves, por lo que termina por demorarse mucho más en realizar la consulta, pues si hubiera filtrado solo a partir de nacionalidad habría ganado, pero al preguntar por otros elementos termina por demorarse mucho más.
totalKeysExamined	0	12	
nReturned (global)	10	10	
nReturned (inputStage)	12	12	
executionTimeMillis	5	29	
Plan usado/elegido	'PROJECTION_SIMPLE'		

Comparación consulta 3:

Campo	Antes	Después	Conclusión
totalDocsExamined	35	35	Ahora se puede ver que revisa muchas más llaves, pero que al hacer uso de otro Plan logra un rendimiento similar (poco superior). Lo más importante es que usa el método LIMIT , el cual filtra por registros/elementos/documentos.
totalKeysExamined	0	35	
nReturned (global)	1	1	
nReturned (inputStage)	1	25	
executionTimeMillis	8	11	
Plan usado/elegido	'PROJECTION_SIMPLE'	LIMIT	

Comparación consulta 4:

Campo	Antes	Después	Conclusión
totalDocsExamined	35	35	Todo igual
totalKeysExamined	0	0	
nReturned (global)	35	35	
CollScan	Igual todo		
Plan usado/elegido	PROJECTION SIMPLE		



executionTimeMillis	2	2	
---------------------	---	---	--

3) Índice sobre Edad

Supondré que es independiente al punto anterior, así que, eliminé el índice de nacionalidad para poder describir y probar realmente cual es el impacto de este índice por sí solo. Se crea el índice con esta línea: **.createIndex({"Edad":1})**

Consulta 1

```
> db["Voice-actors"].find(
  {"Edad":{"$gt":50}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      Edad: {
        '$gt': 50
      }
    },
  },
  indexFilterSet: false,
  queryHash: 'A757D537',
  planCacheShapeHash: 'A757D537',
  planCacheKey: 'E2A0F8A5',
  optimizationTimeMillis: 16,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  prunedSimilarIndexes: false,
  winningPlan: {
    isCached: false,
    stage: 'LIMIT',
    limitAmount: 10,
```

Consulta 2

```
>_MONGOSH
> db["Voice-actors"].find(
  {"Nacionalidad": "Estados Unidos","Idiomas":{"$in: ["Inglés"]}},
  {"_id": 0, "Nombre":1,"Idiomas": 1, "Nacionalidad":1,"Edad": 1})
.sort({"Nombre":1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Idiomas: {
            '$eq': 'Inglés'
          }
        },
        {
          Nacionalidad: {
            '$eq': 'Estados Unidos'
          }
        }
      ]
    },
  },
  indexFilterSet: false,
  queryHash: 'A2EC8B8A',
  planCacheShapeHash: 'A2EC8B8A',
  planCacheKey: '23A50762',
  optimizationTimeMillis: 3,
  maxIndexedOrSolutionsReached: false,
```



Consulta 3

```
> db["Voice-actors"].find(
  {"Edad":{"$lte: 30},"Personajes":{"$elemMatch":{"Rol":"principal", "Géneros":{"$in: ["Drama"]}}} },
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes": 1})
.sort({"Nacionalidad":1})
.limit(10).explain("executionStats")

< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Actors.Voice-actors',
    parsedQuery: {
      '$and': [
        {
          Personajes: {
            '$elemMatch': {
              '$and': [
                {
                  Géneros: {
                    '$eq': 'Drama'
                  }
                },
                {
                  Rol: {
                    '$eq': 'principal'
                  }
                }
              ]
            }
          }
        }
      ]
    }
  }
}
```

Consulta 4

Explain Plan

Explain provides key execution metrics that help diagnose slow queries and optimize index usage. [Learn more](#)

[Visual Tree](#) **Raw Output**

```
{
  "explainVersion": "1",
  "stages": [
    {
      "$cursor": {
        "queryPlanner": {
          "namespace": "Actors.Voice-actors",
          "parsedQuery": {},
          "indexFilterSet": false,
          "queryHash": "13009E3D",
          "planCacheShapeHash": "13009E3D",
          "planCacheKey": "5C6E11E9",
          "optimizationTimeMillis": 0,
          "maxIndexedOrSolutionsReached": false,
          "maxIndexedAndSolutionsReached": false,
          "maxScansToExplodeReached": false,
          "prunedSimilarIndexes": false,
          "winningPlan": {
            "isCached": false,
            "stage": "PROJECTION_SIMPLE",
            "transformBy": {
              "Nombre": true,
              "Patrimonio": true,
              "Edad": true,
              "Nacionalidad": true,
              "_id": false
            }
          },
          "inputStage": {
```

Query Performance Summary

- 10 documents returned
- 35 documents examined
- 2 ms execution time
- Is not sorted in memory
- 0 index keys examined
- No index available for this query.

Close



Después de añadir el índice de edad:

Comparación consulta 1:

Campo	Antes	Después	Conclusión
totalDocsExamined	35	10	Gana en revisar “menos documentos”, pero eso es sobre compensado por lo que revisa de llaves, pues termina demorándose casi 5 veces más, en este caso era mejor no tener el índice de edad.
totalKeysExamined	0	10	
nReturned (global)	10	10	
nReturned (inputStage)	21	10	
executionTimeMillis	5	24	
Plan usado/elegido	PROJECTION_SIMPLE		

Comparación consulta 2

Campo	Antes	Después	Conclusión
totalDocsExamined	35	35	En este caso en específico parece no haber un cambio significativo al crear un índice para edad
totalKeysExamined	0	0	
nReturned (global)	10	10	
nReturned (inputStage)	12	12	
executionTimeMillis	5	5	
Plan usado/elegido	PROJECTION_SIMPLE		

Comparación consulta 3

Campo	Antes	Después	Conclusión
totalDocsExamined	35	1	En este caso en específico solo hay 1 elemento que cumple con lo pedido / filtrado, ya que se está buscando por edad solo revisa 1 documento, pero al buscar la llave parece que se demora al buscar entre todas las edades, o quizás al tener el nuevo índice se demora pues busca por Edad y por su índice.
totalKeysExamined	0	1	
nReturned (global)	1	1	
nReturned (inputStage)	1	1	
executionTimeMillis	8	10	
Plan usado/elegido	PROJECTION_SIMPLE		

Comparación consulta 4

Campo	Antes	Después	Conclusión
totalDocsExamined	35	35	Todo igual

totalKeysExamined	0	0	
nReturned (global)	35	35	
CollScan	Igual todo		
Plan usado/elegido	PROJECTION_SIMPLE		
executionTimeMillis	2	2	

4) Comparación de planes de ejecución

Para la **consulta 1**, se puede obtener que simplemente se demoró más en ambos casos, para el caso del índice de la Nacionalidad se mantiene bastante igual, mientras que con el índice de Edad termina revisando menos documentos, sin embargo, también revisa más llaves, por lo que se compensa y termina haciendo que se demore mucho más de lo esperado. La razón por la que posiblemente se demora más ha de ser debido al plan usado por MongoDB, en el caso sin índice y en los casos con índices es el mismo: **PROJECTION_SIMPLE**, lo cual significa que, filtra por columnas/campos, por lo que, al no cambiar de método, se termina por ejecutar filtrar columna a columna, y en cada documento o valor hay más “columnas” ya que se añadió un nuevo índice, lo cual podría afectar al rendimiento.

Para la **consulta 2**, solo hay un cambio significativo al añadir el índice de Nacionalidad, puesto que, con el índice de Edad no cambia nada. Se puede concluir que con la adición del índice de Nacionalidad se lee menos documentos, pero se usa más llaves, por lo que termina por demorarse mucho más en realizar la consulta.

Analizando la **consulta 3** se pueden ver 2 cosas importantes, para el índice de Nacionalidad hay un cambio de plan, haciendo uso del plan “**LIMIT**”, y que para el índice de Edad se lee 1 documento y 1 llave, teniendo además un tiempo de ejecución similar. Primero, para el índice de Nacionalidad el uso de un nuevo plan por parte de MongoDB puede deberse al **project**, pues es el único lugar donde se usa el campo Nacionalidad además del \$sort, pudiendo verse que revisa 35 documentos y 35 llaves, pero que por el método distinto puede conseguir un rendimiento no peor como en el caso de la consulta 2. Posiblemente esto se deba a que logra “limitar” rápidamente el único caso donde ocurre, decidiendo no leer los demás. Segundo, está buscando por Edad, y como solo hay 1 elemento que lo cumple solo revisa ese, después para los demás filtros revisen que se cumplen y no revisa más archivos, por lo tanto, esta búsqueda resulta bastante buena al revisar menos documentos, y termina teniendo una diferencia de 2 milisegundos con el método sin índices, revisando casi nada, pues limita su búsqueda de inmediato, ayudado por el índice de Edad.

Es importante considerar que para el caso anterior solo había 1 elemento que cumpliera las condiciones, por lo tanto, podría no ser tan relevante dicho cambio de plan.

Finalmente, para la **consulta 4** no se ve ningún cambio para ninguno de los dos índices, simplemente se mantiene igual, al ser estilo pipeline, es una forma de trabajar filtrando en

cada nivel, y que no cambien se debe a que los filtros de este pipe no tienen nada que ver con los índices añadidos.

5) Escalabilidad

En este caso al realizar consultas sobre tan pocos datos no se puede aprovechar al máximo el uso de los índices, en este caso en particular no resulta mucho más ganadero que usar los demás campos para buscar elementos que cumplan ciertas condiciones. **Las consultas ejecutadas sin índices**, son la forma normal de operar, pero al no usar índices me imagino que no podría realizar búsqueda binaria entre campos no numéricos, lo cual sería computacionalmente lo mejor para buscar un elemento a partir de ciertas condiciones de $>$, $<$, $>=$ o $<=$, sin mencionar que si se utiliza la indexación con hash, entonces la velocidad de acceso o lectura podría dispararse, mejorando el rendimiento en comparación a no usar índices. Por lo tanto, sería importante utilizar índices para escalar, suponiendo que tuvieran varias máquinas es posible que cada máquina tenga una partición de elementos/registros, y que, al conocer los índices de cierto valor, ir a buscar a esa máquina resulte mucho más veloz en comparación a buscarlo en una base de datos con todos los valores, beneficiándose de una búsqueda por rangos.

El tema con **las consultas que pueden utilizar los índices creados**, se podría teorizar que mejorarían el rendimiento de algunas consultas, pero si se añaden índices a todas las variables entonces no estaría mejorando mucho. Es clave pensar que el uso de nuevos índices es para agilizar el acceso a ciertos valores en vez de tener que leer todos hasta encontrarlo, y que si se añaden nuevos índices estos tienen que ir pensados por las consultas.

Por índices **la cantidad de documentos que MongoDB tendría que examinar**, disminuiría, puesto que, en las consultas donde se haga uso de los índices se tendrá un acceso más veloz a los documentos, y puede que resulte más “liviano” revisar por índices que por campos normales (como de strings de varias palabras, espacios y demás). De hecho, sí se confirmó que en las consultas donde los índices sí formaban parte importante de ella, que los documentos examinados disminuyó, mientras que las llaves revisadas aumentaron, tal como ocurrió para la consulta 2 para el primer índice y las consultas 1 y 3 para el segundo índice (Nacionalidad y Edad respectivamente).

En teoría al tener más índices, se tendría también más columnas que almacenar y recorrer, por lo que, **el tiempo y recursos necesarios para procesar las consultas**, tendería en aumentar si no se realizan con tanta frecuencia las consultas que hacen uso ideal de los nuevos índices utilizados, lo cual se demuestra en este caso pequeño con la consulta 2 con el uso del índice de Nacionalidad, terminando por demorarse casi 5 veces más que sin el uso de índices, pudiendo generar una demora mucho más grave con una base de datos de 10 mil millones de elementos.

La conclusión a la que se puede llegar en este caso es que, se debe de elegir cuidadosamente los nuevos índices por crear, pues su mal uso causaría una pérdida en **almacenamiento** y en rendimiento, puesto que, **el almacenamiento adicional requerido por los índices**, no se puede ignorar.

A partir del `.explain()`, además de analizar el tiempo tiene mayor importancia el acceso/lectura/revisión/examinación de documentos y llaves/claves, pues son estos los parámetros que pueden hacer que de revisar 10 mil millones de elementos a pasar a revisar solo 1000 por ejemplo. Y tiene importancia porque cada acceso es un ping o llamada a algún elemento de la base de datos para revisar, y si reviso muchos elementos obviamente el proceso tomaría mucho más tiempo, se puede pensar que el acotar o podar el espacio de búsqueda genera una ganancia de tiempo relevante.

Parte 3

1. Flexibilidad de esquema

Contexto: <<Suponga que se inserta un nuevo actor o actriz de voz, pero el documento no contiene el atributo “Fecha de nacimiento”.>>

¿MongoDB permitiría insertar el documento?

R: Sí, y los demás campos no los consideraría para la búsqueda, es posible además rellenar los campos vacíos como null en la sección Aggregations de MongoDB compass. Por ejemplo, al añadir mi nombre y edad ocurre lo siguiente (no explota):

```
> db["Voice-actors"].insertOne({"Nombre":"Felipe Parra","Edad":21})
< {
  acknowledged: true,
  insertedId: ObjectId('6aa4b425d8b44e966e6cf9d0')
}
```

Y esto se muestra normal en las consultas, por ejemplo, aquí busco por edad personas menores de 50 años y sorteo de menor a mayor, y aparece mi registro sin problema con los campos que tiene nomás. En caso de que fuera necesario se puede rellenar con vacío los demás campos, pero independiente de eso, sí puedo insertar un documento al que le falten campos.

```
> db["Voice-actors"].find(
  {"Edad":{"$lt":50}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10)
< {
  Nombre: 'Felipe Parra',
  Edad: 21
}
{
  Nombre: 'Aleks Le',
  Nacionalidad: 'Estados Unidos',
  Edad: 27,
  Personajes: [
    {
      Nombre: 'Mikey',
      'Serie, película, anime o videojuego': 'Tokyo Revengers',
      'Tipo de producción': 'profesional',
      Rol: 'principal',
      'Cantidad de apariciones': 50,
      'Géneros': [
        'Acción',
        'Drama',
        'Artes marciales'
      ]
    }
  ],
}
```

¿Qué consecuencias podría tener posteriormente?

R: Que si no coloco la fecha de nacimiento no pueda compararse por fechas de nacimiento, aunque si podría hacer operaciones del tipo “que no tenga esta fecha de nacimiento” terminando por salir en cada caso, continuando el ejemplo anterior, al filtrar por un campo que no tiene simplemente no aparecerá o no será considerado, pero para consultas de negación aparecería igual:

Este ejemplo es el mismo al anterior, pero ahora reviso que no sea español la persona y me muestra a mí, porque no tengo nacionalidad en los registros, por lo tanto, cumpliendo la condición de “no ser español”.

```
> db["Voice-actors"].find(
  {"Edad":{$lt:50},"Nacionalidad":{$ne: "España"}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10)

< {
  Nombre: 'Felipe Parra',
  Edad: 21
}
{
  Nombre: 'Aleks Le',
  Nacionalidad: 'Estados Unidos',
  Edad: 27,
  Personajes: [
    {
      Nombre: 'Mikey',
      'Serie, película, anime o videojuego': 'Tokyo Revengers',
      'Tipo de producción': 'profesional',
      Rol: 'principal',
      'Cantidad de apariciones': 50,
      'Géneros': [
        'Acción',
        'Drama',
        'Artes marciales'
      ]
    }
  ]
}
```

¿Considera conveniente permitir documentos con atributos faltantes en este caso?

R: Sí, es conveniente, aquí no gastamos columnas vacías como en bases de datos relacionales, al ser documentales simplemente tengo elementos con menos “columnas” o campos, por lo que, se podría realizar una lectura más rápida de dichos registros y no se generarían problemas. Reduciría el almacenamiento requerido para esos registros y si dichos atributos faltantes fueran realmente importantes, podría añadirse luego y no habría problema.

Me añado y me busco:

```
> db["Voice-actors"].updateOne({"Nombre": "Felipe Parra"},
  {$set: {"Nacionalidad": "Chile"}}
)

< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
> db["Voice-actors"].find(
  {"Edad":{$lt:50},"Nacionalidad":{$ne: "España"}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10)

< {
  Nombre: 'Felipe Parra',
  Edad: 21,
  Nacionalidad: 'Chile'
}
{
  Nombre: 'Aleks Le',
  Nacionalidad: 'Estados Unidos',
  Edad: 27,
  Personajes: [
    {
      Nombre: 'Mikey',
      'Serie, película, anime o videojuego': 'Tokyo Revengers',
      'Tipo de producción': 'profesional',
      Rol: 'principal',
      'Cantidad de apariciones': 50,
      'Géneros': [
        'Acción',

```



¿Existe alguna forma de controlar o validar la estructura de los documentos en MongoDB?

R: Sí, como mencioné anteriormente se podría hacer uso de un pipeline con aggregate donde rellenara los campos vacíos con null en caso de que fuera necesario, esto se puede realizar con \$fill, ejemplo corto donde rellene a cualquier registro que le falte cierto campo con cierto valor:

Tarea-1-BD-Av > Actors > Voice-actors Open MongoDB shell

Documents 36 Aggregations Schema Indexes 1 Validation

Project Ssort Sfill Generate aggregation Explain Run Options

Untitled - modified SAVE CREATE NEW EXPORT DATA EXPORT CODE PREVIEW STAGES TEXT WIZARD

1 /**
2 * Provide any number of field/order pair
3 */
4 {
5 Edad: 1
6 }

Output preview after Ssort stage (Sample of 10 documents)

Nombre: "Felipe Parra" Edad: 21 Nacionalidad: "Chile"	Nombre: "Aleks Le" Nacionalidad: "Estados Unidos" Fecha de nacimiento: "1999/06/12" Edad: 27 Patrimonio: 500000 Idiomas: Array (1) Personajes: Array (2)
---	--

Stage 3 Sfill ON

1 {
2 sortBy: {Edad: 1},
3 output: {
4 "Fecha de nacimiento": {value: "2000/
5 }
6 }
7 }

Output preview after Sfill stage (Sample of 10 documents)

Nombre: "Felipe Parra" Edad: 21 Nacionalidad: "Chile" Fecha de nacimiento: "2000/01/01"	Nombre: "Aleks Le" Nacionalidad: "Estados Unidos" Fecha de nacimiento: "1999/06/12" Edad: 27 Patrimonio: 500000 Idiomas: Array (1) Personajes: Array (2)
--	--

2. Resiliencia de la base de datos

Botando el nodo primario:

```
root@aca01138dc5d:/# mongosh -port 30001
Current Mongosh Log ID: 6aa4bbb46a5b59a40fa3ac8b
Connecting to:      mongodb://127.0.0.1:30001/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:      8.0.30
Using Mongosh:       2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-09-11T22:19:33.740+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-11T22:19:33.742+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-11T22:19:33.742+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-11T22:19:33.742+00:00: We suggest setting the contents of sysfsFile to 0.
2026-09-11T22:19:33.743+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems
.
-----
my-replica-set [direct: primary] test> []
```

Como decía “primary” es fue el que boté con las instrucciones del enunciado de la tarea

Al hacer: **rs.stepDown(60)**

Termina el mongo2 como primary

```
Problems Output Debug Console Terminal Ports SonarQube

name: 'mongo2:30002',
health: 1,
state: 1,
stateStr: 'PRIMARY',
uptime: 16318,
optime: { ts: Timestamp({ t: 1789181488, i: 2 }), t: Long('2') },
optimeDurable: { ts: Timestamp({ t: 1789181488, i: 2 }), t: Long('2') },
optimeWritten: { ts: Timestamp({ t: 1789181488, i: 2 }), t: Long('2') },
optimeDate: ISODate('2026-09-12T02:51:28.000Z'),
optimeDurableDate: ISODate('2026-09-12T02:51:28.000Z'),
optimeWrittenDate: ISODate('2026-09-12T02:51:28.000Z'),
lastAppliedWallTime: ISODate('2026-09-12T02:51:28.381Z'),
lastDurableWallTime: ISODate('2026-09-12T02:51:28.381Z'),
lastWrittenWallTime: ISODate('2026-09-12T02:51:28.381Z'),
lastHeartbeat: ISODate('2026-09-12T02:51:36.882Z'),
lastHeartbeatRecv: ISODate('2026-09-12T02:51:36.387Z'),
pingMs: Long('0'),
lastHeartbeatMessage: '',
syncSourceHost: '',
syncSourceId: -1,
infoMessage: '',
electionTime: Timestamp({ t: 1789181488, i: 1 }),
electionDate: ISODate('2026-09-12T02:51:28.000Z'),
configVersion: 1,
configTerm: 2
```

Y si realizo una consulta en la terminal de MongoDB Compass, se ejecuta normal como si no hubiera pasado nada. La consulta funciona pues se esta usando la arquitectura maestro-esclavo, por lo que, al bajar el nodo primario, los nodos secundarios eligen quien será el nuevo nodo primario y vuelve a un estado “estable”, pudiendo realizar consultas de lo más normal y sin fallos.

3.Actualización masiva de documentos

No alcance a terminar esta parte

```
> db["Voice-actors"].find(
  {"Edad":{$lt:50},"Nacionalidad":{$ne: "España"}},
  {"_id": 0, "Nombre":1,"Edad": 1, "Nacionalidad":1,"Personajes":1})
.sort({"Edad": 1})
.limit(10)

< {
  Nombre: 'Felipe Parra',
  Edad: 21,
  Nacionalidad: 'Chile'
}
{
  Nombre: 'Aleks Le',
  Nacionalidad: 'Estados Unidos',
  Edad: 27,
  Personajes: [
    {
      Nombre: 'Mikey',
      'Serie, película, anime o videojuego': 'Tokyo Revengers',
      'Tipo de producción': 'profesional',
      Rol: 'principal',
      'Cantidad de apariciones': 50,
      'Géneros': [
        'Acción',
        'Drama',
        'Artes marciales'
      ]
    }
  ]
}
```



FIN

No olvidar que todo el registro de los datos se encuentra en el github además de las carpetas dentro de la entrega final.

Link de GitHub: <https://github.com/ShadierPrune/BD-Av-Tarea-1-MongoDB.git>